



Documents Fonctionnels :
PBS et WBS

Historique des versions

Version	Par qui ?	Quand ?	Quoi ?	Pourquoi ?
1.0	Alexia Legrand	24/01/24	Tout	Création du document
1.1	Alexia Legrand	05/02/24	PBS et WBS	Corrections
2.0	Jalil Jaajoui	12/06/24	Mise en page	Application Charte Graphique
3.0	Jalil Jaajoui	10/01/25	Réduction du scope	Réduction du scope

Sommaire

Introduction.....	4
1) [Début document].....	5



Introduction

Plock est un réseau social où l'utilisateur peut jouer à des mini-jeux créés par la communauté. Les membres de la communauté ont la possibilité de concevoir des jeux à l'aide d'un éditeur directement intégré à l'application. Toutefois, pour une expérience plus approfondie, une version web de l'éditeur est disponible, offrant davantage d'options de personnalisation.

PBS



Description du PBS (fonctionnalités du projet)

Plock est une application mobile, inspirée de Tiktok en remplaçant la vidéo par des minis jeux sympa créés par la communauté, pour la communauté. L'idée est de créer une plateforme communautaire autour des jeux vidéos, les utilisateurs peuvent jouer aux jeux des autres, les liker, les commenter, les partager.

Nous avons l'ambition de permettre aux utilisateurs de construire leur propre jeu avec un système de création simple, mais complet, inspiré de ce que peut faire Unity ou Godot.

Note : Toutes les captures d'écrans disponibles dans ce document sont le fruit d'un POC (proof of concept) et/ou de montages et/ou de mockups et ne représentent en rien le résultat final.

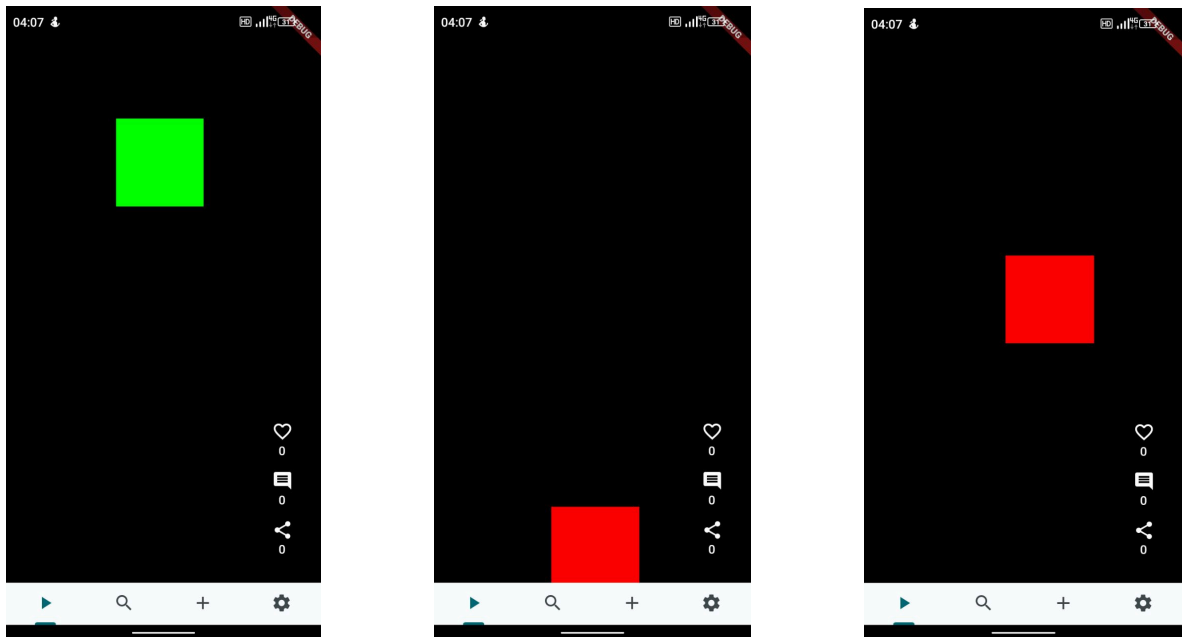
1) User player

1.1) Réseau Social

Création d'un réseau social complet où les médias seront les jeux eux-mêmes. Les utilisateurs pourront "liker" les jeux, les commenter et les partager. Chaque utilisateur aura son propre profil sur lequel il retrouvera l'intégralité des jeux créés.

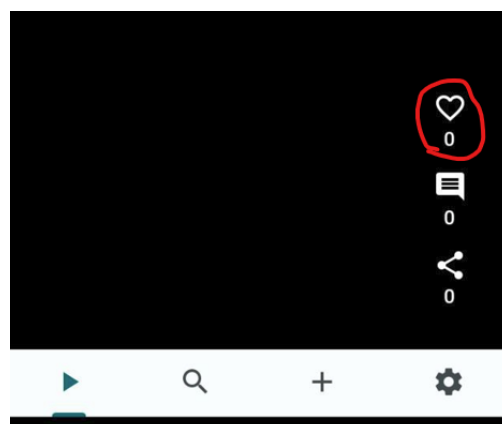
1.1.1) Swipe

A la manière de tiktok, les jeux passeront de l'un à l'autre avec un système de swipe, ou lors de la victoire du jeu. Le flux des jeux est infini, tout du moins jusqu'à ce qu'il n'y ai plus de jeux jouables. (exemple d'un swipe ci-dessous, où l'on passe du jeu du carré vert au jeu du carré rouge)



1.1.2) J'aimes

Les utilisateurs peuvent aimer un jeu, ce qui améliorera son référencement et permettra aux joueurs de retrouver le jeu sur leurs profils, dans la catégorie 'j'aime'.



1.2) Compte

La gestion des comptes utilisateurs est une part importante des réseaux sociaux et se doit d'être efficace et sécurisée.

1.2.1) Connexion

Tout système utilisateur nécessite une page de connexion pour pouvoir accéder au contenu de l'application.

1.2.2) Inscription

L'utilisateur doit pouvoir s'inscrire à l'application pour commencer à l'utiliser.

1.2.3) Edition du profil

L'utilisateur doit pouvoir modifier son profil : pseudo, email, mot de passe, avatar,

1.3) Algorithme

Comme tout réseau social, notre app devra implémenter un algorithme de suggestion de contenu basé sur les préférences utilisateurs pour toujours afficher des jeux qui captivent l'utilisateur.

1.3.1) Catégoriser les jeux

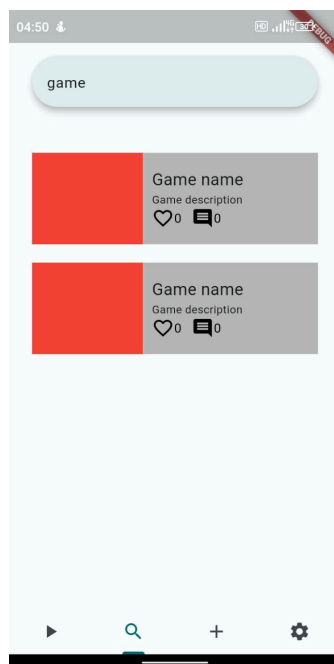
Pour pouvoir faire des recherches ou de la recommandation, il faut pouvoir catégoriser les types de jeux, en utilisant des tags par exemple.

1.3.2) Recommendations

Les recommandations sont vitales dans un réseau social car elles définissent ce que l'utilisateur va consommer. Il faut un set de jeux recommandés (en fonction des tags des jeux likés par le compte)

1.3.3) Recherche

Les utilisateurs peuvent rechercher un jeu spécifique à l'aide du titre ou des tags pour trouver (ou retrouver) un jeu spécifique.



2) Éditeur

L'éditeur sera le second grand volet de l'application, cette partie permettra aux utilisateurs de créer simplement et facilement leur jeux. La création de jeux passera par un système simple et ingénieux d'objets sur lesquels nous y appliqueront des composants. La philosophie du game editor se veut totalement no code afin de permettre même au débutant de l'explorer

2.1) Système d'objets

L'éditeur de jeux fonctionne avec un système d'objets et de composants. L'utilisateur peut créer des objets (par exemple un mur) et lui appliquer des composants (par exemple le fait de bloquer le joueur pour qu'il ne traverse pas le mur) afin de créer son jeu.

2.1.1) Affichage des objets

La première étape de l'éditeur est de pouvoir ajouter et voir des objets, qu'ils aient un sprite ou non.

2.1.2) Édition des objets

Ensuite, les objets doivent pouvoir être édités : les nommer, les déplacer.

2.1.3) Ajouter des composants

Enfin, chaque objet peut être assigné des composants pour définir leurs comportements dans le jeu. Un même objet peut avoir plusieurs composants.

2.2) Composants

Chaque objet peut avoir des composants, tels que de la gravité, une boîte de collision, un sprite, ... qui eux même ont des propriétés (force de la gravité, apparence du sprite, ...). Cela permet de donner aux objets toutes les propriétés qui font de l'objet un joueur, un mur, un ennemi, ...

2.2.1) Edition des composants

Les composants peuvent être édités pour correspondre mieux au but du créateur, et varient en fonction du composant (un composant de gravité pourra changer sa force de gravité, un sprite pourra changer d'apparence, ...)

2.2.2) Liens inter composants

Les composants peuvent être reliés entre eux pour plus d'interactions, par exemple un composant de victoire en cas de contact avec un autre objet spécifique.

2.2.3) Liste de composants

la quantité de composants peut être infini, et il est important de bien remplir le catalogue de composants, sans pour autant perdre les nouveaux utilisateurs

2.3) Publier

Les jeux enfin créés sont prêts à être publiés et joués par le reste de la communauté, mais il faut encore passer quelques étapes.

2.3.1) Test

Les jeux doivent être testés avant d'être publiés, pour s'assurer qu'ils sont terminables et qu'ils ne durent pas plus de 5 minutes.

2.3.2) Metadata des jeux

Les jeux contiennent des données tel que leur nom et leur descriptions. Ces metadata doivent être défini par l'utilisateur avant la publication du jeu

2.3.3) Publication des jeux

Le jeu peut enfin être publié, il est envoyé sur les serveurs qui ensuite l'ajoute à la liste des jeux jouables.

3) Afficheur

L'afficheur permet de jouer aux jeux, il affiche, applique les comportement spécifié par le créateur et permet les interactions pour jouer.

3.1) Afficher des objets

L'afficheur doit pouvoir afficher les objets définis d'un jeu.

3.2) Application des composants

Les objets créés devront réagir en accord avec les composants qui lui sont assignés. Si un objet a le composant gravité, alors cet objet doit être soumis à la physique. La liste et les effets des composants sont diverses, de activer la victoire jusqu'à une caméra qui se déplace dans le niveau.

3.3) Interactions

Les interactions, telles que la victoire, un glisser déposer, un clic, ... devront être gérées par l'afficheur pour que le jeu soit jouable.



4) Base de donnée / API

4.1) Stockage des jeux

Les jeux doivent être stockés en ligne pour ensuite pouvoir les envoyer aux différents joueurs de l'application.

4.1.1) Format des jeux

Il est primordial de définir la structure du fichier qui sauvegarde un jeu, pour pouvoir le créer dans l'éditeur, le stocker dans la base de donnée et l'interpréter dans l'afficheur.

4.1.2) Librairie des jeux

Il faut pouvoir sauvegarder une grande quantité de jeux, et les envoyer aux différents joueurs sans qu'il y ait trop de latence et que la base de données sature.

4.2) Social

L'aspect social (comptes utilisateurs, likes,...) se doit également d'être sauvegardé dans la base de données.

4.2.1) Gérer les comptes

Les comptes devront être sauvegardés et sécurisés sur la base de données.



Description du WBS

1) Début

Le premier objectif est de mettre en place les outils et les projets, de créer une base sur laquelle ajouter du contenu.

1.1) Mockups

Les mockups servent à visualiser l'application et que tout le monde ait le même référentiel visuel.

Requis :

- Mockup de l'afficheur
- Mockup de l'éditeur

1.2) Structure des jeux

Il faut définir sous quel format les jeux seront sauvegardés et partagés.

Requis :

- Document définissant la structure des jeux

1.3) Application basique

La base de l'application : une navbar qui mène vers l'éditeur ou l'afficheur ainsi que la possibilité de faire glisser l'écran (swiper) pour changer de jeu



Requis :

- Une navbar avec 2 tab
- la première tab qui permet d'accéder à l'afficheur
- la seconde tab qui permet d'accéder à l'éditeur
- La possibilité de 'swiper' d'un jeu à l'autre

1.4) Afficheur basique

L'afficheur doit être capable de lire un fichier de jeu et de l'interpréter.

Requis :

- Possibilité de lire un fichier de jeu
- Capable d'interpréter un composant d'affichage d'un rectangle coloré

1.5) Éditeur basique

L'éditeur doit être capable de créer un jeu et d'exporter un fichier de jeu (cette phase n'intègre pas encore la publication et la récupération des jeux en ligne).

Requis :

- Créer un objet
- Déplacer un objet dans l'éditeur
- Ajouter un composant aux objets
- Éditer le composant
- Avoir un composant qui permet d'afficher un rectangle coloré
- Exporter le jeu



1.6) Stockage des jeux

Mise en place d'une API et d'une base de donnée qui permet de stocker les jeux

Requis :

- Avoir une base de donnée qui peut stocker les jeux
- Mise en place de l'intégration continue
- Création d'une API qui permet de sauvegarder et de récupérer des jeux

2) Composants

Cette deuxième phase a pour but de lier toutes les parties créées dans la première phase ainsi que d'ajouter des composants et de l'interactivité pour enfin avec un contenu jouable.

2.1) Contrôleurs

Il est temps d'ajouter de l'interaction en permettant l'usage de contrôleurs

Requis :

- Contrôleur avec déplacement sur la gauche et interactions sur la droite (style manette)
- glisser déposer des objets
- cliquer sur un objet

2.2) Gravité

La gravité est un composant important dans beaucoup de jeux vidéo, notamment les jeux de plateforme.

Requis :

- Gestion de la gravité
- Composant rigidbody
- Composant static objects

2.3) Mécanique de victoire

Un jeu se termine lorsque le joueur 'swipe' ou lorsqu'il gagne le jeu.

Requis :

- Création de composants permettant plusieurs type de victoires différentes

2.4) Lien des composants

Les composants peuvent parfois interagir avec d'autres composants ou d'autres objets.

Requis :

- Pouvoir lier un composant en éditant un autre composant
- Avoir une interface intuitive (cliquer sur l'objet sur la scène, ou sur une liste avec tous les objets).



2.5) Tester dans l'éditeur

Lorsqu'on crée un jeu, il est important de pouvoir le tester avant de le publier.

Requis :

- Pouvoir tester le jeu dans l'éditeur sans le publier
- Tester que le jeu dure moins de 5 minutes, sinon bloquer la publication.

2.6) Publication des jeux en ligne

Il est temps de lier l'application et l'api.

Requis :

- Possibilité de publier des jeux plutôt que de les exporter
- L'application peut désormais lire des jeux à l'infinie (ou jusqu'à la fin des jeux disponibles dans la base de données)

3) Social

Cette troisième phase consiste à ajouter toute la dimension sociale à l'application, que ce soit la création de compte ou les interactions sociales.

3.1) Gestion des utilisateurs

Les utilisateurs peuvent créer des comptes et des jeux

Requis :

- Pouvoir s'inscrire avec son compte google
- Pouvoir se connecter avec son compte google
- Réserver la création des jeux aux personnes inscrites
- Pouvoir visualiser et éditer ses jeux

3.2) Édition des profils

L'utilisateur doit pouvoir éditer ses informations personnelles.

Requis :

Page de profil

éditer son pseudo

éditer son mail

éditer son avatar

3.3) J'aime

Le joueur doit pouvoir aimer les jeux qu'il apprécie en cliquant sur un bouton

Requis :

- Le joueur peut mettre un 'j'aime' sur les jeux qu'il apprécie

4) Algorithmes

Cette phase permet de mettre en place la recherche et l'algorithme de recommandation de l'application.

4.1) Métadonnées des jeux

Le joueur doit désormais remplir un rapide formulaire avant de publier son jeu.

Requis :

- Choisir un titre
- Remplir une description
- Ajouter des tags

4.2) Recherche

Un onglet de recherche est désormais ajouté pour rechercher un jeu spécifique.

Requis :

- Créer un nouvel onglet de recherche
- ajouter un algorithme de recherche
- afficher une liste de jeux correspondant à la recherche
- Lancer le jeu lorsqu'on clique dessus

4.3) Recommendations

Les recommandations sont réalisées à l'aide d'un algorithme.

Requis :

- Définir le comportement de l'algorithme
- Avoir un algorithme de recommandation fonctionnel